Exercises for
Introduction to Machine Learning
Linda-Sophie Schneider, Utkarsh Bansal, Md Abdul Ahad,
Keyi Hou, Max Zernickel, Yumeng Zhang
Task 3: deadline: 14 June 2026

Pattern Recognition Lab

Exercise 03: HOG Features for Kaktovik Symbols

Exercise 1 Research-Oriented Exercise

In this exercise, you will work in a way that is common in many research projects: you will read a paper, identify the relevant parts for your task, and implement a simplified but faithful version of the method.

The paper for this exercise is the original HOG paper by Dalal and Triggs [1], which is provided on StudOn and in the exercise folder as `histogramsOfOrientedGradients.pdf`. You are **not** supposed to implement the complete detection pipeline from the paper. Instead, you should focus on the descriptor itself and apply it to our handwritten Kaktovik symbols.

Exercise 2 Alignment, HOG Feature Extraction, and Distance Comparison

In this exercise, you will work with our Kaktovik symbol dataset. The goal is to combine **preprocessing** and **feature extraction**: first align the symbols spatially, then compute HOG features, and finally compare different symbols using both a pixel-based baseline and a feature-based distance.

A code skeleton is provided consisting of the following files:

- `main_kaktovik.py`: main file for loading images, running the pipeline, and printing comparisons (**do not modify**).
- `kaktovikAlignmentSimple.py`: module for simple spatial alignment.
- `HOGFeature.py`: module for computing HOG features.
- `DistanceMeasure.py`: module for the comparison measures.
- `test_Ex3.py`: tests for your implementation.
- `img/retrieval/`: a retrieval gallery with 10 provided dataset images per symbol class from our processed Kaktovik dataset.

Implementation policy: In this exercise, you should code as much as possible yourself. You may use `NumPy` for array operations, slicing, masking, aggregation, reshaping, and basic mathematical functions. You may use `OpenCV` only for low-level image operations such as image loading, resizing, thresholding, geometric transformations, and Sobel filtering. You must **not** use ready-made high-level feature extraction or similarity functions that already solve the task for you.

Your task consists of four parts:

Part 1 - Simple Alignment

Complete the function `simpleAlignment(img, size=128)` in `kaktovikAlignmentSimple.py`. The aim of this part is to normalize the position and size of a symbol before feature extraction. Your aligned output should contain the centered symbol on a fixed square canvas. You may use low-level image operations from `OpenCV`, but the actual localization and placement logic should be implemented by yourself using `NumPy`. In particular, do not use contour-based or other high-level object localization routines.

Part 2 - HOG Feature Extraction

Complete `HOGFeature.py`. Based on [1], implement a simplified dense HOG descriptor for our aligned symbol images. The descriptor should be based on image gradients, orientation histograms on local cells, and block-wise normalization. For comparability across submissions, use cells of size 8×8, blocks of size 2×2 cells, and 9 unsigned orientation bins. The provided main file uses the functions `computeGradients(...)`, `buildCellHistograms(...)` and `calculateHOG(...)`; do not change these names or their arguments.

Important: You are expected to implement the descriptor yourself. You may use `cv2.Sobel(...)` for the derivatives, but do **not** use `cv2.HOGDescriptor`, `skimage.feature.hog`, or any other ready-made HOG implementation.

Part 3 - Distance Measures

Complete `DistanceMeasure.py` by implementing the **mean squared error (MSE)** between two equally sized grayscale images and the **Euclidean distance** between two feature vectors. Here, `NumPy` is allowed for elementwise arithmetic, reductions, and norms, but you should still implement the formulas explicitly in your own code. Do not use external metric libraries.

Use these distances to compare aligned symbols, for example identical symbols, the same symbol written by different students, the same symbol under different rotations, and different symbols.

Part 4 - Robustness and Retrieval

Use the provided `main_kaktovik.py` to generate translated and rotated variants of a symbol, align them, and compare their distances. Then use the folder `img/retrieval/`, which contains 10 provided dataset images for each symbol class in the gallery, and evaluate a simple retrieval setup:

- Treat each image once as a query.
- Rank the remaining gallery images by similarity.
- Report top-1, top-3, and top-5 retrieval accuracy for both HOG and MSE.
- Use aligned images throughout this evaluation.
- For a few translated or rotated query images, inspect how the retrieval ranking behaves after alignment.

Images that share the same prefix (for example `class7_...`) belong to the same retrieval class. The exact transformation parameters used in the dataset are not important; the goal is to evaluate the resulting descriptor behavior.

Interpret the results: Which geometric changes are already compensated by the preprocessing? How does MSE behave compared to HOG? Where does the descriptor remain sensitive, even after alignment? How does retrieval behave on

artificially transformed queries compared to the already processed gallery?

Expected Discussion

This exercise is intentionally not about classification yet. Instead, you should understand how a feature descriptor behaves on real data, why preprocessing matters, why a feature-based comparison can be more meaningful than raw pixel comparison, why robustness to translation does not automatically imply robustness to rotation, and how the computed features can already be used for a simple retrieval task before any classifier is trained.

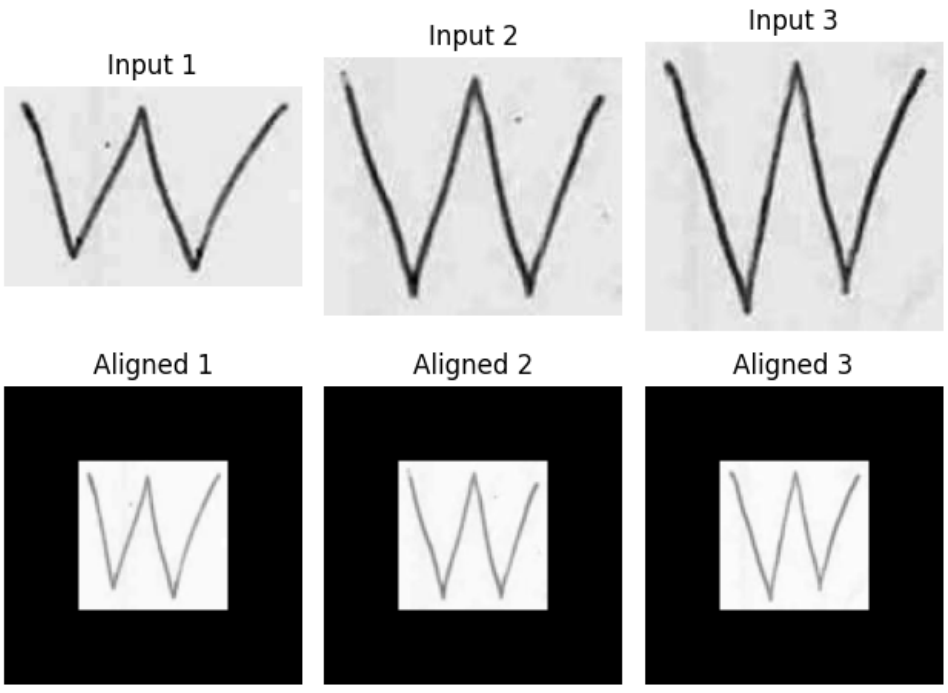
Important Hint 1: Do not modify the main file. If you want to run additional experiments, create a copy.

Important Hint 2: The tests are written to be path-independent. Your code should therefore also avoid assumptions about the current working directory when loading images in scripts.

Important Hint 3: Do not modify or extend the provided retrieval gallery if you want your results to stay comparable to the tests and to the other submissions. The gallery intentionally contains multiple dataset images per class, not only transformed copies of a single prototype.

Important Hint 4: If you are unsure whether a helper function is too high-level, ask yourself: does this function only perform a basic image or array operation, or does it already solve a substantial part of the exercise for me? In the latter case, do not use it.

Important Hint 5: Here is an example of how the aligned symbols should roughly look:



---

[1] Navneet Dalal and Bill Triggs. "Histograms of oriented gradients for human detection." In *Proceedings of the IEEE Computer Society Conference on Com-*

puter Vision and Pattern Recognition (CVPR), 2005.